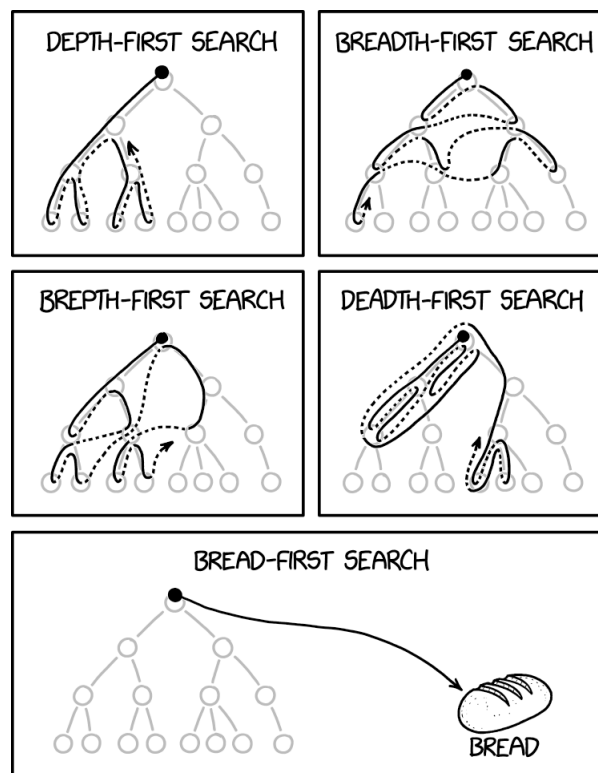


Chapitre 3

Union-Find



1 Structure abstraite

On va ici présenter une structure permettant de représenter une partition d'un ensemble fini.

Soit E un ensemble fini, une partition de E est une famille $\{A_i\}_{i \in I}$ de parties de E deux à deux disjointes dont la réunion est E tout entier. Ces parties sont appelées classes car l'on sait que ce sont les classes d'équivalence d'une certaine relation d'équivalence.

On va définir une représentation des partitions de l'ensemble $\{0, \dots, n-1\}$ (tout ensemble fini de cardinal n peut être mis en bijection avec cet ensemble) muni des opérations suivantes :

1. Création d'une nouvelle partition de taille n où chaque élément est dans sa propre classe : `create : int -> partition`
2. Union des classes de deux éléments de E passés en argument : `union : partition -> int -> int -> unit`
3. Recherche de la classe associée à un élément de E : `find : partition -> int -> int`

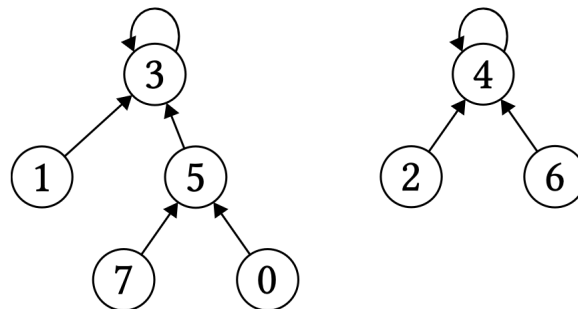
Un appel à `create` sur l'entrée n construit alors une nouvelle partition de $\{0, \dots, n-1\}$ où chaque élément forme une classe à lui tout seul.

L'opération `find` appliquée à une partition et un entier i détermine la classe de l'élément i dans cette structure, sous la forme d'un entier considéré comme l'**unique représentant** de cette classe. En particulier, on détermine si deux éléments sont dans la même classe en comparant les résultats donnés par `find` pour chacun.

Enfin, l'opération `union` réunit les deux classes des éléments passés en argument, la structure de données étant modifiée en place.

2 Une première implémentation

L'idée principale est de lier entre eux les éléments d'une même classe. Dans chaque classe, ces liaisons forment des chemins qui mènent tous à un unique représentant, qui est le seul élément lié à lui-même. La figure ci-dessous montre un exemple où l'ensemble $\{0, 1, \dots, 7\}$ est partitionné en deux classes dont les représentants sont respectivement 3 et 4.



Il est donc facile de matérialiser ces relations par un simple tableau qui relie chaque entier à un autre entier de la même classe. Ces liaisons mènent toujours au représentant de la classe, qui est associé à sa propre valeur dans le tableau. Ainsi, la partition de la figure est représentée par le tableau suivant : $\{5, 3, 4, 3, 4, 3, 4, 5\}$.

Réalisons alors notre structure en OCAML. On considère la variable `uf` qui est un tableau d'entiers représentant une partition comme décrit ci-dessus.

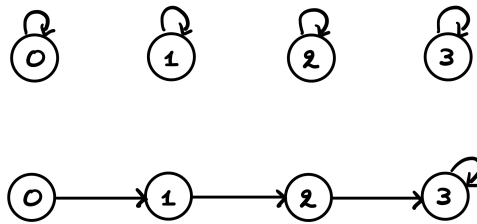
```
1 let create (n : int) = Array.init n (fun i -> i)
2
3 let rec find uf i =
4   if uf.(i) = i then i
5   else
6     let p = uf.(i) in
7     find uf p
8
9 let union uf i j =
10  uf.(i) <- uf.(j)
```

On remarque que si i et j étaient dans la même classe, alors l'union n'aurait eu aucun effet.

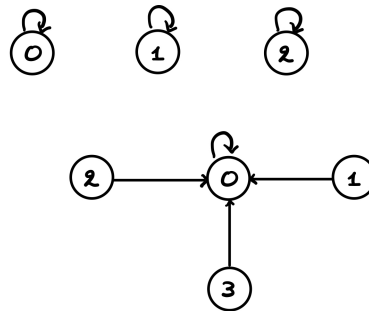
On pouvait difficilement imaginer un code plus simple que cela. Cependant, notre structure est un peu naïve. En effet, on peut se retrouver avec de très longues chaînes dans le tableau, voire impliquant les n éléments. C'est le cas par exemple si on fait `union uf i (i+1)` pour tout i . Dès lors, la complexité de `find` et de `union` peuvent être aussi grandes que $\mathcal{O}(n)$. Ce n'est pas acceptable en pratique...

EXEMPLES.

1. Illustrer l'évolution de la partition avec 4 éléments initialement isolés si on effectue `union uf i (i+1)` pour tout `i` :



2. Illustrer l'évolution de la partition avec 4 éléments initialement isolés si on effectue `union uf (i+1) i` pour tout `i` :



➤ Fort heureusement, il est facile d'atteindre de bien meilleures performances, en apportant deux améliorations à notre code.

3 Optimisation première : union pondérée

On remarque que lors de l'union, on fait pointer le représentant d'une des deux classes vers le représentant de l'autre. Le choix des rôles n'est, en réalité, pas arbitraire puisqu'une classe peut contenir des chemins beaucoup plus longs que l'autre. Ainsi, on va maintenir, pour chaque représentant, une valeur appelée rang qui représente la longueur maximale que pourrait avoir un chemin dans cette classe. Cette information est stockée dans un second tableau, à côté du tableau qui contient les liaisons.

```
1 type uf = {
2   link : int array;
3   rank : int array;
4 }
```

L'information contenue dans `rank` n'est significative que pour des éléments `i` qui sont des représentants, i.e tels que `link.(i) = i`. Initialement, le rang de chaque classe vaut 0.

On en déduit donc la fonction `create` suivante :

```
1 let create n =
2   { link = Array.init n (fun i-> i) ; rank = Array.make n 0 }
```

Le rang est ensuite utilisé par la fonction `union` pour choisir entre les deux représentants possibles d'une union.

```
1 let union uf i j =
2   let ri = find uf i in
3   let rj = find uf j in
4   if (ri <> rj) then
5     begin
6       if uf.rank.(ri) < uf.rank.(rj) then uf.link.(ri) <- rj
7       else if uf.rank.(ri) < uf.rank.(rj) then uf.link.(rj) <- ri
8       else
9         begin uf.rank.(ri) <- uf.rank.(ri) + 1 ; uf.link.(rj) <- ri end
10    end
```

Et cette optimisation est déjà intéressante!

Théorème

Une classe de rang k possède des chemins de longueur maximale k et au moins 2^k éléments.

DÉMONSTRATION.

On fait la preuve par récurrence sur le nombre d'appels à **union**.

- Si $n = 0$: la classe est de hauteur 0 et de taille $1 = 2^0$.
- Supposons qu'après n opérations, la propriété soit vérifiée. Considérons une opération supplémentaire.
 - Si on fait l'union de la classe **i** de représentant **ri** de rang k et de la classe **j** de représentant **rj** de rang k' , avec $k < k'$.
 Ainsi, **ri** ne sera plus représentant mais **rj** le sera de nouveau avec toujours ce rang k' . Par H.R, la classe de **j** avait déjà au moins 2^k éléments. Cette propriété reste donc vérifiée.
 Par ailleurs, en ce qui concerne un chemin enraciné en **rj** :
 - Soit il passe par **ri** et donc de hauteur $\max \leq k + 1$, i.e $\leq k'$.
 - Soit il ne passe pas par **ri** et donc (par H.R) il est de hauteur au plus k' .
 - Analogie si $k > k'$
 - Enfin, si on fait une union entre 2 classes de même rang k alors le rang du représentant de la nouvelle classe obtenue est $k + 1$, la classe obtenue est donc au moins de cardinal $2 \times 2^k = 2^{k+1}$ par H.R. Si un chemin depuis le représentant passe par l'autre représentant : il aura une hauteur maximale qui vaut $k + 1$ s'il ne passe pas par cet ancien représentant, la hauteur maximale sera donc k par H.R.

d'où le résultat voulu.

Proposition - Complexité des opérations due à l'optimisation première

Grâce à l'optimisation première, on obtient des opérations **find** et **union** en $\mathcal{O}(\log(n))$.

DÉMONSTRATION.

Si on a une classe de rang k alors elle a au moins 2^k éléments, ainsi $2^k \leq |E| \Rightarrow k = \mathcal{O}(\log_2 |E|)$.

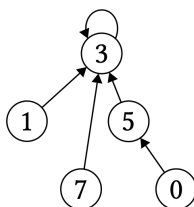
Or, les opérations **find** et **union** ont une complexité en $\mathcal{O}(h_{\max} \text{ d'une classe }) = \mathcal{O}(\log_2 |E|)$.

d'où la complexité recherchée.

$\Rightarrow$ On peut cependant encore améliorer l'efficacité de notre structure!

4 Optimisation seconde : compression des chemins

L'idée consiste à compresser les chemins pendant la recherche de **find**. En fait, on tire profit du parcours de la branche pour compresser le chemin et ainsi obtenir plus tard de meilleures complexités (on entre donc dans le domaine de la complexité amortie). Ainsi, on lie directement au représentant de la classe tous les éléments rencontrés sur le chemin parcouru pour l'atteindre lors de l'appel à **find**. Par exemple, un appel à **find** **uf** 7 dans la situation de la figure précédente va renvoyer 3 et modifier la structure pour que 7 pointe désormais vers 3.



On obtient alors la nouvelle fonction **find** suivante :

```

1 let rec find uf i =
2   let p = uf.link.(i) in
3   if (p=i) then i
4   else begin
5     let r = find uf p in
6     uf.link.(i) <- r;
7     r
8   end

```

En particulier, après le calcul de $r = \text{find uf } i$, on a $\text{uf.link.}(i) = r$ directement. Mais on a aussi $\text{uf.link.}(j) = r$ pour tous les éléments de j qui se trouvaient initialement sur le chemin entre i et r grâce aux appels récursifs.

Il est important de noter que la fonction `union` utilise la fonction `find` et réalise donc des compressions de chemin, même dans le cas où il s'avère que i et j sont dans la même classe. Notre théorème de la partie précédente reste valable dans la mesure où l'on entend « de longueur maximale k » comme « de longueur au plus k ». En effet, une classe peut avoir le rang k mais aussi des chemins, par conséquent, tous strictement plus petits que k car la compression de chemin les a tous raccourcis. En particulier, on a donc toujours une complexité au pire logarithmique pour les opérations `find` et `union`.

Proposition - Complexité des opérations due à l'optimisation seconde

La complexité amortie des deux opérations est en $\mathcal{O}(\alpha(n))$.

où α est l'inverse de la fonction d'Ackermann.

Cette fonction croît si lentement qu'on peut la considérer comme constante pour toute application pratique (au vu des valeurs limites de n que la mémoire nous autorise à entrer, c'est pas délirant), ce qui nous permet de supposer un **temps amorti constant** pour chaque opération.

⇒ Cette analyse de complexité est subtile et dépasse largement le cadre du programme...

Exercice

Implémenter la structure union-find en C avec la structure ci-dessous.

```

1 struct UnionFind{
2     int size; int* link; int* rank;
3 };
4 typedef struct UnionFind uf;

1 uf* create(int size){
2     uf* res = malloc(sizeof(uf)); res->rank = malloc(size* sizeof(int)); res->link = malloc(size * sizeof(
3         int));
4     for (int i=0; i<size; i++){
5         (res->link)[i] = i;
6         (res->rank)[i] = 0;
7     }
8     res->size = size;
9     return res;
10 }

11 int uf_find(uf* uf, int i){
12     if ((uf->link)[i] = i){
13         return i;
14     }
15     else{
16         int p = (uf->link)[i];
17         int r = find(uf, p);
18         (uf->link)[i] = r;
19         return r;
20     }
21 }

22 void uf_union(uf* uf, int i, int j){
23     int ri = uf_find(uf,i); int rj = uf_find(uf,j);
24     if (ri != rj){
25         int rank_ri = (uf->rank)[ri]; int rank_rj = (uf->rank)[rj];
26         if (rank_ri > rank_rj){
27             (uf->link)[rj] = ri;
28         }
29         else if (rank_rj > rank_ri){
30             (uf->link)[ri] = rj;
31         }
32         else{
33             (uf->link)[rj] = ri; (uf->rank)[ri] += 1;
34         }
35     }
36 }
37 }
```

5 Bonus : la fonction d'Ackermann

La fonction d'Ackermann-Péter est définie **récurivement** comme suit :

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1, 1) & \text{si } m > 0 \text{ et } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{si } m > 0 \text{ et } n > 0 \end{cases}$$

On remarque que **cette fonction croît extrêmement rapidement**